

RAG: What Nobody Tells You When Building Your AI Assistant

Everyone says RAG is easy until you actually try to build one.



Photo by [Uday Mittal](#) on [Unsplash](#)

When I first heard about Retrieval-Augmented Generation (RAG), it sounded like magic. None members can read it [here](#)

You feed it your documents, ask questions, and it gives smart, personalized answers using an LLM. Simple, right?

Except... it wasn't.

Behind the buzzwords and polished demos, building a working RAG system had me Googling non-stop, questioning my embeddings, and debugging why my chatbot kept saying “*I don't know*”.

This article is the honest guide I wish existed when I started. Not just how to build RAG but what nobody tells you when you're getting started.



Image by author — Accurate RAG output

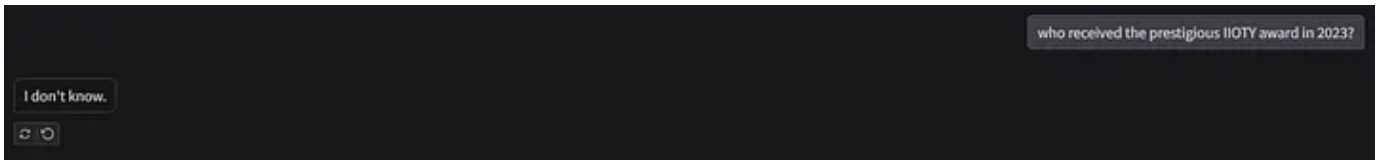


Image by author — Inaccurate RAG output

. . .

Quick refresher: What is RAG?

RAG is a framework that combines two powerful ideas:

1. **Retrieval:** Pull relevant chunks of external data (like PDFs, docs, or wikis) at runtime.
2. **Generation:** Feed those chunks into a language model (like GPT-4 or Mistral) to generate a contextual answer.

To make this retrieval precise and meaningful, vector encoding is used. When a user asks a question, the question is transformed into a vector, that is a numerical representation that captures its meaning, known as an embedding. Then this vectorized query is compared with the document vectors stored in a specialized database. Each of these vectors represents pieces of text from our knowledge base. By finding vectors closest in meaning, the model is sure to retrieve only the most relevant information. That retrieved text is combined with the user's question and fed to the LLM, empowering it to generate answers grounded in real, contextual knowledge.

Here's a simplified flow:

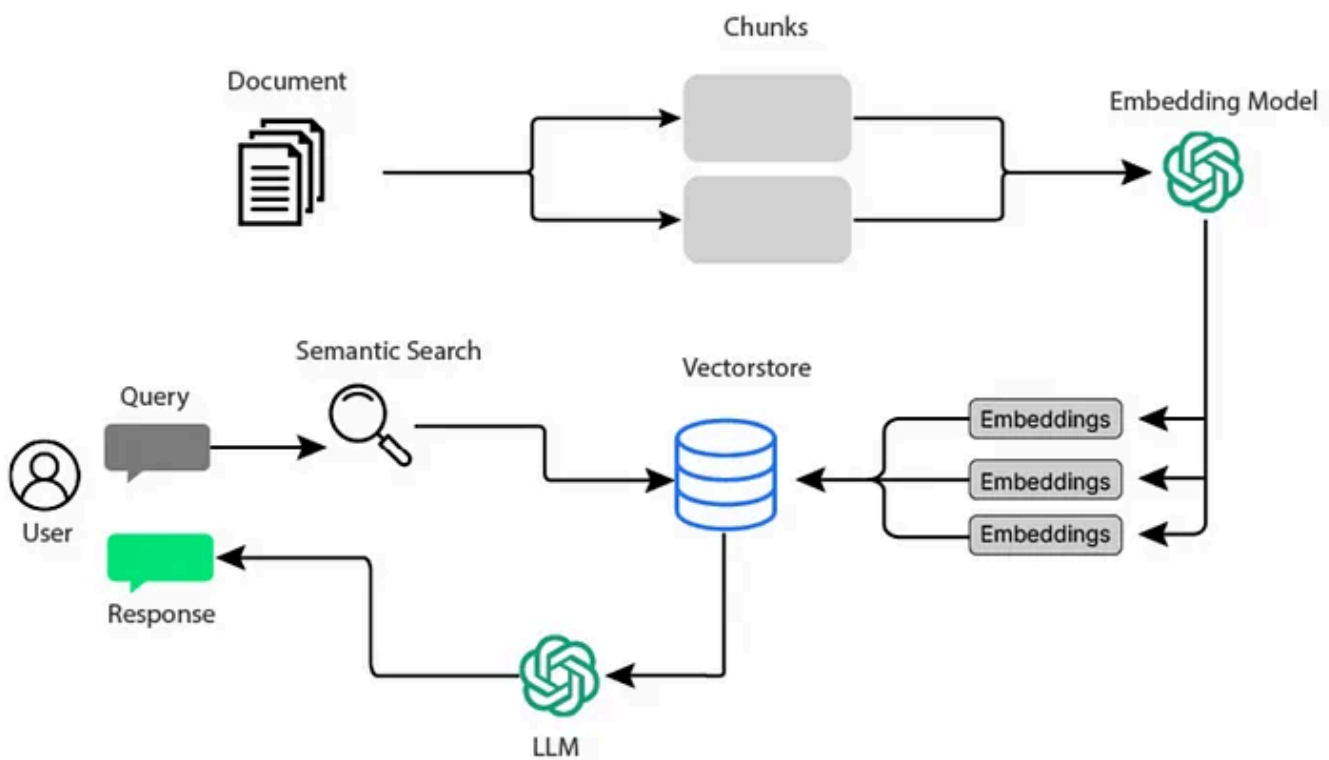


illustration by [deepsandyha](#)

. . .

What Nobody Tells You About Building a RAG System

1. Chunking is everything

Your document is not fed completely into the LLM, it is first split into chunks, and each chunk is vectorized, and the meaning of the chunk is stored in a vector store to be retrieved when needed by the LLM.

I started with arbitrary chunks, every 500 words. Big mistake.

Turns out, how you break up your documents directly affects what gets retrieved. A badly chunked document is like trying to learn from shredded newspaper, although, technically, the info is there, but good luck making sense of it.

Poor chunking = irrelevant context = bad answers.

Here's a quick example:

- ❌ *Bad Chunk*: "The RAG architecture is popular today. Retrieval-Augmented Generation..."
- ✅ *Good Chunk*: "The RAG architecture is popular today because it retrieves external context at runtime to enhance generation. This approach combines information retrieval with large language models."

💡 **Tip:** Use semantic chunking or sentence-based splits. Tools like LangChain and LlamaIndex make this easier, but experimentation is key.

2. All embeddings are not created equal

Embeddings are vector representations of text that allow models to find semantically similar content, not just keyword matches.

For a long time, I was blaming my vector DB for bad results but the issue was my embedding model. Switching from `text-embedding-ada-002` to `bge-small-en` made my results 2× more relevant.

Some models do better with technical docs, some with plain text, and some are just faster. Your embedding model affects both **search accuracy** and **latency**.

💡 **Tip:** Try multiple embedding models and **visualize** your vector space using tools like `TensorBoard` or `plotly`. Sometimes it's not your retriever, it's your embeddings.

3. Retrieval isn't plug-and-play

RAG uses vector database to store and retrieve embeddings based on similarity. There are two commonly used ones: Chroma and FAISS (Facebook AI similarity search).

I started with FAISS (because everyone does), but quickly ran into limitations such as no filtering, limited persistence support, scaling issues. Tools like Pinecone or Weaviate offer more flexibility (but with costs).

I switched to Chroma but its not free of limitations either; its slower than FAISS, and my system keeps crashing when I use smaller chunk size for my documents. Hence, this limits the chunking and overlapping flexibility.

But the number of documents you retrieve (`k`) can overwhelm your LLM or make the RAG miss key context.

💡 **Tip:** Tune your retriever. Try different values of `k`, use similarity thresholds, and monitor relevance quality.

When to go local: For Rapid prototyping, no filtering, small-scale use.

When to go hosted: Metadata search, production scaling, and real users.

4. LLMs have short attention spans

Even GPT-4 has a max token limit (even with 128k context, it's expensive). Stuff too many chunks into the prompt and the model gets confused or truncates important data.

💡 **Tip:** Rank chunks by relevance, merge small ones, and be selective about what goes into the context window.

5. Latency is a real UX problem

Your pipeline includes: text splitting → embedding → DB search → LLM generation.

This worked on my laptop, but after incorporating a UI, it starts to lag.

Latency adds up. If you're building something user-facing:

- Preprocess embeddings

- Cache recent results
- Batch queries where possible

💡 **Tip:** Consider local vector DBs like FAISS or Chroma during dev. Use async calls. And cache results where possible.

6. Evaluation is messy

How do you know if your RAG system is *good*?

There's no single accuracy metric like in classification tasks. You need to evaluate:

- Retrieval quality
- Faithfulness of answers
- Hallucination rate
- User satisfaction

💡 **Tip:** Log everything. Manually review results. Use tools like RAGAS or LangChain's eval module once you're confident with the basics.

. . .

📁 If I Were Starting Today

- I will pick a **smaller dataset** to iterate quickly
- I will focus on **chunk quality before model tuning**
- I will use **local embeddings and vector DBs** in dev to save cost
- I will add **citations in output** from the beginning (users trust them)

. . .

🧑 Final Thoughts

RAG is one of the most powerful techniques in the LLM toolbox especially when you're working with private or domain-specific data.

But it's not magic out of the box.

It takes iteration, thoughtful design, and a bit of trial-and-error to go from “why is this broken?” to “wow, it actually works.”

If you're getting started, I hope this gave you the head start I wish I had.